In a similar way, `wordwrap()` also inserts newlines but works a little differently. As you can see in the previous example, `chunk_split()` does not care about word boundaries and will simply insert the newline sequence in the middle of a word (in this case, `"string"` was transformed into `"st\r\nring"`). `wordwrap()` by default will keep "long" words (words that would otherwise have newlines inserted into them) intact:

```
print wordwrap('A string of text', 4);
```

```
Output:
A
string
of
text
```

And because `wordwrap()` is aware of word boundaries, it can handle UTF-8 data safely. The example below shows multi-byte (in fact, multi-codepoint) characters in a string processed with a width value of `1`: Observe that the flag glyphs emerge unscathed. (Your mileage may vary with other encodings; be warned and test thoroughly!)

```
print wordwrap("\u{1F3F3}\u{FE0F}\u{200D}\u{1F308} \u{1F3F4}\u{200D}\
u{2620}\u{FE0F}", 1);
```

```
Output:
```



As before, the "bald man" caveat applies: if your system cannot parse/render multi-codepoint glyphs correctly, then your output will differ.

`nl2br()`

Another function with its roots in the early internet is `nl2br()`, which inserts the HTML `<br>` tag (or the XHTML `<br />` tag) in front of every newline character in a string. Web browsers reduce all whitespace and newline characters to a single whitespace, and this trick makes newline sequences (`CRLF`, `LFCR`, `LF`, and `CR`) behave in the browser as they would in, say, a terminal window.